

## Operações comuns sobre uma base de dados

- ▶ Pesquisa
- ▶ Inserção
- ▶ Alteração
- ▶ Eliminação

## Exemplo:

Base de Dados : Vinhos

Tabela: Garrafas

| Código | Vinho | Produtor | Ano | Qt |
|--------|-------|----------|-----|----|
| -----  | ----- | -----    | --- | -- |
| 1      | Tinto | ACRM     | 90  | 20 |
| 2      | Douro | CVPR     | 91  | 40 |
| 3      | Verde | CVPL     | 90  | 35 |

### Exemplo: Pesquisa em SQL

```
Select Vinho, Código, Produtor
From Garrafas
Where Ano = 90;
```

O resultado será:

| Vinho | Código | Produtor |
|-------|--------|----------|
| ----- | -----  | -----    |
| Tinto | 1      | ACRM     |
| Verde | 3      | CVPL     |

### Exemplo: Inserção em SQL

```
Insert into Garrafas
Values (53, 'Porto', 'Sandeman',82, 20);
```

```
Select *
From Garrafas;
```

| Código | Vinho | Produtor | Ano | Qt |
|--------|-------|----------|-----|----|
| -----  | ----- | -----    | --- | -- |
| 1      | Tinto | ACRM     | 90  | 20 |
| 2      | Douro | CVPR     | 91  | 40 |
| 3      | Verde | CVPL     | 90  | 35 |
| 53     | Porto | Sandeman | 82  | 20 |

### Exemplo: Alteração em SQL

```
Update Garrafas
Set Qt = 4
where código = 3;
```

```
Select *
From Garrafas;
```

| Código | Vinho | Produtor | Ano | Qt |
|--------|-------|----------|-----|----|
| -----  | ----- | -----    | --- | -- |
| 1      | Tinto | ACRM     | 90  | 20 |
| 2      | Douro | CVPR     | 91  | 40 |
| 3      | Verde | CVPL     | 90  | 4  |
| 53     | Porto | Sandeman | 82  | 20 |

### Exemplo: Eliminação em SQL

```
Delete From Garrafas
Where código = 3;
```

```
Select *
From Garrafas;
```

| Código | Vinho | Produtor | Ano | Qt |
|--------|-------|----------|-----|----|
| -----  | ----- | -----    | --- | -- |
| 1      | Tinto | ACRM     | 90  | 20 |
| 2      | Douro | CVPR     | 91  | 40 |
| 53     | Porto | Sandeman | 82  | 20 |

## Pesquisa de Dados com SQL

- ▶ Pode ser feita através da instrução SELECT (query)
  - ▶ **interactivo**, o operador digita:
 

```
Select city
From S
Where S# = 'S4';
```
  - ▶ **Forms** nas aplicações cliente
  - ▶ SQL embebido na linguagem do SGBD do servidor
 

```
EXEC SQL SELECT city
          INTO : XCity
          FROM S
          WHERE S# = 'S4';
```

: indicam que se segue uma variável da linguagem hospedeira.  
**Xcity** é a variável da host language que recebe o resultado.

▶ 46

46

## Estrutura de Dados Relacional

- ▶ Pode dividir-se um modelo relacional em três partes:
  - ▶ Estrutura de dados.
  - ▶ Integridade dos dados.
  - ▶ Manipulação dos dados.
- ▶ Todas as partes têm os seus termos especiais e vamos falar dos termos que dizem respeito à estrutura de dados.
- ▶ **Relação** é um termo matemático cuja teoria foi aplicada por Dr. Codd, nas bases de dados relacionais.

▶ 47

47

## Termo Relacional e Equivalente Informal

| Termo Relacional | Equivalente Informal           | Descrição                                         |
|------------------|--------------------------------|---------------------------------------------------|
| Relação          | Tabela                         | Conjunto de tuplos com atributos definidos        |
| Tuplo            | Linha, registo                 | Representa uma ocorrência ou instância da relação |
| Atributo         | Coluna, campo                  | Representa uma propriedade ou característica      |
| Domínio          | Conjunto de valores permitidos | Ex.: domínio do atributo “idade” pode ser [0–120] |
| Grau             | Número de atributos            | Ex.: 4 atributos → grau 4                         |
| Cardinalidade    | Número de tuplos               | Ex.: 10 linhas → cardinalidade 10                 |

## Operações Relacionais

- No modelo relacional, as operações que manipulam relações derivam da **álgebra** e **cálculo relacional**:

| Operação                        | SQL equivalente  | Descrição                       |
|---------------------------------|------------------|---------------------------------|
| Seleção ( $\sigma$ )            | SELECT ... WHERE | Filtra tuplos                   |
| Projeção ( $\pi$ )              | SELECT colunas   | Seleciona atributos             |
| Junção ( $\bowtie$ )            | JOIN             | Combina tuplos de duas relações |
| União ( $\cup$ )                | UNION            | Combina conjuntos               |
| Diferença ( $-$ )               | EXCEPT, MINUS    | Diferença entre dois conjuntos  |
| Produto cartesiano ( $\times$ ) | FROM A, B        | Todas as combinações possíveis  |

## Chaves e Integridade

### ▶ Chave Primária (Primary Key)

- ▶ Atributo(s) que **identificam univocamente** cada tuplo da relação.
- ▶ Deve ser **único** e **não nulo**.
- ▶ Deve ser o número mínimo de atributos

### ▶ Chave Estrangeira (Foreign Key)

- ▶ Atributo(s) de uma relação que **referem uma chave primária** noutra.
- ▶ Garante **integridade referencial** entre relações.
- ▶ Relação referente → a que contém a chave estrangeira.
- ▶ Relação referenciada → a que contém a chave primária de valor igual à chave estrangeira.

### ▶ Chaves Candidatas e Alternativas

- ▶ **Chaves candidatas**: todas as combinações possíveis que poderiam funcionar como chave primária.
- ▶ **Chaves alternativas**: chaves candidatas que **não foram escolhidas** como chave primária.
- ▶ Uma chave composta pode incluir dois ou mais atributos.

▶ 51

51

## Propriedades de uma Chave Primária/Candidata

### ▶ Unicidade:

- ▶ Em qualquer ocasião não há dois tuplos com o mesmo valor de chave primária/candidata.

### ▶ Minimalidade:

- ▶ Se a chave é composta, ao eliminar-se um dos atributos da chave a unicidade é destruída.

### ▶ Integridade da entidade

- ▶ Uma chave primária não pode ter valores nulos (NULL), isto é tem de ter informação.

▶ 52

52

## Integridade Relacional

- ▶ As regras de integridade são fundamentais para assegurar que a base de dados **reflete corretamente a realidade modelada**.
- ▶ **Integridade de entidade**: a chave primária **não pode ser nula**.
- ▶ **Integridade referencial**: os valores de uma chave estrangeira devem:
  - ▶ Ser nulos (se não fizer parte da chave primária), ou
  - ▶ Existir como chave primária na tabela referenciada.

▶ 53

53

## Exemplo de Modelo Relacional

- ▶ **Relações:**
- ▶ Aluno(NMEC, Nome, Email, CodCurso)
- ▶ Curso(CodCurso, NomeCurso)
  - ▶ **Chave Primária de Aluno**: NMEC
  - ▶ **Chave Primária de Curso**: CodCurso
  - ▶ **Chave Estrangeira de Aluno**: CodCurso → referencia a tabela Curso, cuja Chave Primária é CodCurso
- ▶ O atributo CodCurso da tabela Aluno deve referir um curso existente em Curso

▶ 54

54

## Representação de Relações em SQL

```
CREATE TABLE Curso (  
    CodCurso CHAR(3) PRIMARY KEY,  
    NomeCurso VARCHAR(100));  
  
CREATE TABLE Aluno (  
    NMEC INT PRIMARY KEY,  
    Nome VARCHAR(100),  
    Email VARCHAR(100),  
    CodCurso CHAR(3),  
    FOREIGN KEY (CodCurso) REFERENCES Curso(CodCurso));
```

## Exemplo - Tabela S

Tabela S (Fornecedores)

| S# | Sname | Status | City   |
|----|-------|--------|--------|
| -- | ----- | -----  | -----  |
| S1 | Smith | 20     | London |
| S2 | Jones | 10     | Paris  |
| S3 | Blake | 30     | Paris  |
| S4 | Clark | 20     | London |
| S5 | Adams | 30     | Athens |
|    |       |        | S# PK  |

### Exemplo – Tabela P

Tabela: P (Peças)

| P# | Pname | Color | Weight | City   |
|----|-------|-------|--------|--------|
| -- | ----- | ----- | -----  | -----  |
| P1 | Nut   | Red   | 12     | London |
| P2 | Bolt  | Green | 17     | Paris  |
| P3 | Screw | Blue  | 17     | Rome   |
| P4 | Screw | Red   | 14     | London |
| P5 | Cam   | Blue  | 12     | Paris  |
| P6 | Cog   | Red   | 19     | London |

P# PK

### Exemplo – Tabela SP

Tabela: SP (Fornecimentos)

| S# | P# | QTY |
|----|----|-----|
| -- | -- | --- |
| S1 | P1 | 300 |
| S1 | P2 | 200 |
| S1 | P3 | 400 |
| S1 | P5 | 100 |
| S1 | P6 | 100 |
| S2 | P1 | 300 |
| S2 | P2 | 400 |
| S3 | P2 | 200 |
| S4 | P2 | 200 |
| S4 | P5 | 400 |

S#+P# PK  
S# FK S e P# FK P

## Especificação Inicial

### ▶ Restrições:

- ▶ Cada fornecedor está localizado exactamente numa cidade.
- ▶ Cada tipo de peça tem exactamente uma cor e está armazenado exactamente numa cidade.
- ▶ Só pode existir um envio num dado instante de um dado fornecedor e de uma dada peça.

▶ 59

59

## Exemplo – Criação de tabelas com SQL

S<S#, Sname, status, city>; P<P#, Pname, Color, Weight, city>;  
 SP<S#, P#, QTY>, S# FK S, P# FK P

```
CREATE TABLE S (
    S# VARCHAR2(2) CONSTRAINT pk_S# PRIMARY KEY,
    Sname VARCHAR2(20), status NUMBER(2),
    city VARCHAR2(20));
```

```
CREATE TABLE P (
    P# VARCHAR2(2) CONSTRAINT pk_P# PRIMARY KEY,
    Pname VARCHAR2(20), Color VARCHAR2(10),
    Weight NUMBER(2), city VARCHAR2(10));
```

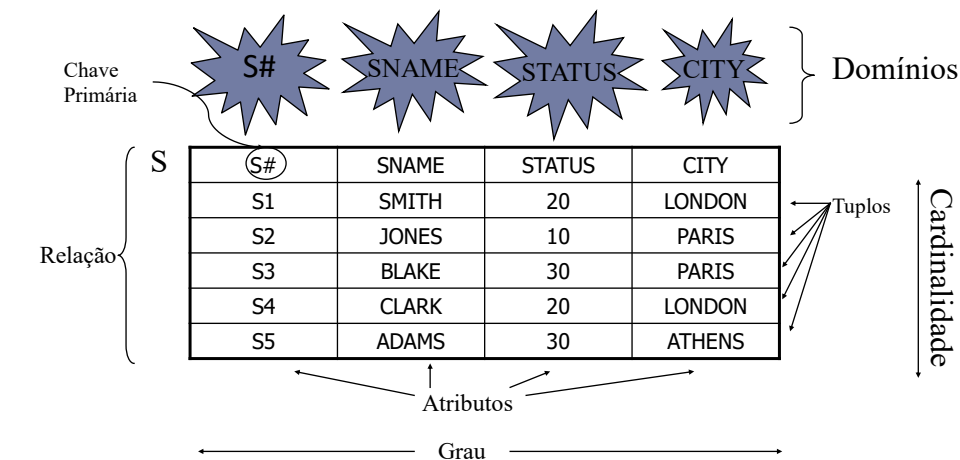
```
CREATE TABLE SP (
    S# VARCHAR2(2),
    P# VARCHAR2(2),
    QTY Number(4),
    CONSTRAINT codsp PRIMARY KEY(S#,P#),
    CONSTRAINT cod_S FOREIGN KEY (S#) REFERENCES S (S#),
    CONSTRAINT cod_P FOREIGN KEY (P#) REFERENCES P (P#));
```

▶ 60

60

## Estrutura de Dados Relacional

2025/2026 - Bases de Dados - Fernanda Brito Correia



▶ 61

61

## Relação, tabela e valor atômico

2025/2026 - Bases de Dados - Fernanda Brito Correia

- ▶ **Relação** é um termo matemático cuja teoria foi aplicada por Dr. Codd, nas bases de dados relacionais
- ▶ Uma relação e uma tabela não são a mesma coisa embora na prática é frequentemente conveniente pretender que o sejam.
- ▶ Um **valor atômico** é a menor unidade semântica de informação. Semântica porque envolve compreensão do contexto onde está inserida.

▶ 62

62

## Requisitos de uma Relação

- ▶ Para que uma estrutura possa ser considerada uma **relação válida**, deve obedecer a:
  - ▶ Todos os valores das células são **atômicos** (não compostos).
  - ▶ Todas as linhas são **únicas** (não podem ser duplicadas).
  - ▶ A ordem das linhas e colunas é **irrelevante**.
  - ▶ Cada atributo tem um **nome único**.
  - ▶ Os atributos têm **valores pertencentes a domínios bem definidos**.
- ▶ Exemplo de violação do modelo: guardar múltiplos valores num mesmo atributo (ex.: lista de telefones numa só célula).

▶ 63

63

## Valor Atômico e Forma Normal (1/3)

- ▶ Numa tabela de uma base de dados relacional, os valores são **atômicos e explícitos**
- ▶ Um valor **atômico** é a menor unidade semântica de informação. Semântica porque envolve compreensão do contexto onde está inserida.
- ▶ O modelo relacional exige que os valores sejam **atômicos**, ou seja, que cada célula armazene apenas **um único valor indivisível**.

| S# | P#           | Errado                          |
|----|--------------|---------------------------------|
| S2 | (P1, P2)     | Se a base de dados é relacional |
| S3 | (P1, P4, P5) |                                 |

▶ 64

64

### Valor Atómico e Forma Normal (2/3)

| S# | P# |
|----|----|
| S2 | P1 |
| S2 | P2 |
| S4 | P1 |
| S4 | P4 |
| S4 | P5 |

Não há ponteiros de umas tabelas para outras visíveis, não quer dizer que não existam ao nível físico.

Correcto

### Valor Atómico e Forma Normal (3/3)

- ▶ Outro exemplo:
- ▶ ✖ Incorreto (violação da 1FN):

| ID | Nome | Telefone             |
|----|------|----------------------|
| 1  | Ana  | 910000001, 930000002 |

- ▶ ✔ Correto:

| ID | Nome | Telefone  |
|----|------|-----------|
| 1  | Ana  | 910000001 |
| 1  | Ana  | 930000002 |

## Noção de Domínio e Semântica

- ▶ A noção de **domínio** em particular serve para ilustrar o ponto importante de que nem todos os sistemas relacionais suportam todos os aspectos do modelo relacional.
- ▶ Um **domínio** é o conjunto de valores possíveis para um atributo. As DBS actuais não implementam domínios. Estes são importantes porque dão sentido ( $\Rightarrow$  compreensão) da parte do DBS.
- ▶ Os domínios **devem ser respeitados semanticamente**.



67

## Importância do Conceito de Domínio

- ▶ A noção de **domínio** em particular serve para ilustrar o ponto importante de que nem todos os sistemas relacionais suportam todos os aspetos do modelo relacional.
- ▶ Um **domínio** é o conjunto de valores possíveis para um atributo.
- ▶ Os domínios são importantes porque dão sentido ( $\Rightarrow$  compreensão) da parte do DBS.



68

68

## Exemplo 1

```
S <S#, SNAME, STATUS, CITY>
P <P#, PNAME, COLOR, WEIGHT, CITY>
SP <S#, P#, QTY>, S# fk S, P# FK P
```

```
SELECT P.*, SP.*
FROM   P, SP
WHERE  P.P# = SP.P#;
```



Faz sentido.  
O DBMS poderia aceitar esta pesquisa.

► 69

69

## Exemplo 2

```
S <S#, SNAME, STATUS, CITY>
P <P#, PNAME, COLOR, WEIGHT, CITY>
SP <S#, P#, QTY>, S# fk S, P# FK P
```

```
-- Exemplo inválido semanticamente
SELECT P.*, SP.*
FROM   P, SP
WHERE  P.WEIGHT = SP.QTY;
```



A DBMS não deveria aceitá-la, porque pesos e quantidades são ambos números mas semanticamente são diferentes (têm sentidos diferentes).  
Este tipo de erro pode não ser detetado pelo SGBD, mas **viola a lógica da aplicação**.

► 70

70

## Exemplo 3

- ▶ Em cada **domínio** certas operações podem não fazer sentido
- ▶ Considere o **domínio** Sexo (Masculino, Feminino), as operações de comparação:
  - ▶ Igual, e
  - ▶ Diferentefazem sentido.
- ▶ Mas por exemplo não faz sentido:
  - ▶  $<=$ ,  $>=$ , ...

▶ 71

71

## Exemplo 4

- ▶ Faz sentido que entre os **domínios** existam certos relacionamentos. Por exemplo:
  - ▶ O domínio “velocidades” multiplicado pelo domínio “tempos” dará o domínio “distâncias”.
- ▶ Notar que todos estes valores são por exemplo reais embora pertençam a **domínios** diferentes.
- ▶ É isto que significa o termo semântico (compreensão do significado, sentido).

▶ 72

72

## Regras de Integridade (1/2)

- ▶ Qualquer base de dados consiste nalguma configuração dos valores dos dados que “reflete a realidade”
  - ▶ um modelo é a representação de uma parte do mundo real
  - ▶ o mundo real surge num instante particular
- ▶ A definição de base de dados deve incluir estas restrições do mundo real, de modo a prever que configurações de valores impossíveis ocorram
  - ▶ A restrição de que o peso de uma peça não pode ser negativo
  - ▶ O DBMS monitoriza as operações INSERT e UPDATE na relação P, das peças, de modo a rejeitar qualquer operação que tente introduzir um valor negativo para P.WEIGHT, o atributo onde é guardado o peso da peça).

▶ 73

73

## Regras de Integridade (2/2)

- ▶ A maior parte das bases de dados estão sujeitas a um grande número de **regras de integridade**.
- ▶ A maior parte das **regras de integridade** são específicas no sentido de que se aplicam a uma base de dados específica.

▶ 74

74

## Arquitetura de uma Base de Dados

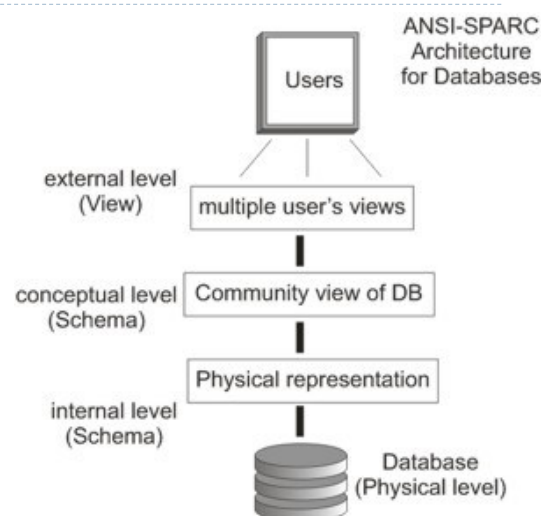
- ▶ *American National Standards Institute, Standards Planning And Requirements Committee (ANSI-SPARC)*
- ▶ É um *design* abstrato, *standard*, para um SGBD proposto em 1975.
- ▶ A maioria dos SGBD comerciais modernos baseiam-se nesta arquitetura
  - ▶ este modelo nunca se tornou num *standard* formal.

▶ 75

75

## Três Níveis de Arquitectura

- ▶ **Nível Externo**
  - ▶ Percepção dos utilizadores
- ▶ **Nível Conceptual**
  - ▶ Estabelece a ligação entre os níveis interno e o externo
- ▶ **Nível Interno**
  - ▶ Descreve o modo como os dados são memorizados nos dispositivos.

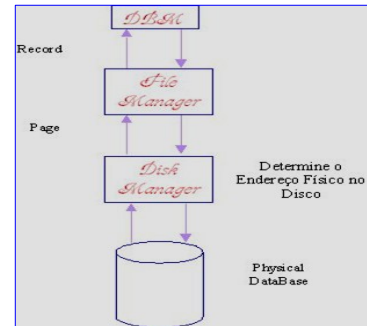


▶ 76

76

## O Nível Interno

- ▶ Sequência de operações numa operação de pesquisa:
  - ▶ O **DBMS** pede ao **File Manager** os *records* necessários
  - ▶ O **File Manager** determina as páginas que contém este *record* e pede-as ao **Disk Manager**;
  - ▶ O **Disk Manager** determina a localização da página em termos de pistas/ sectores do disco;
- ▶ Esta estrutura permite:
  - ▶ **Independência de funções** - estrutura em camadas
  - ▶ Optimizar a disposição física dos dados no disco, deixando juntos (tanto quanto possível) os registos que contém informação relacionada, mesmo entre tabelas distintas → **Clustering**.



▶ 77

77

## Nível Externo (1/4)

- ▶ Três tipos de *interface* com os utilizadores:
  - ▶ O Database Administrator (**DBA**) utiliza comandos próprios do DBMS ou do Sistema Operativo.
  - ▶ O **programador** que usa uma linguagem para desenvolver aplicações. As linguagens da:
    - ▶ 1ª Geração – Código de Máquina
    - ▶ 2ª Geração – Assembler
    - ▶ 3ª e 4ª Geração – C, C#, PHP, Python, R, ...
  - ▶ **Utilizadores finais** usam aplicações *Menu-Driven* ou *Form-Driven* ou uma linguagem de interrogação da base de dados.

▶ 78

78

## Nível Externo (2/4)

### ▶ Linguagens usadas:

- ▶ **Host language** é uma linguagem de programação de 3ª ou de 4ª geração que contém um processo de *interface* com uma Data SubLanguage (DSL).
  - ▶ Exemplo
    - C#, PHP, ....
- ▶ **Embedded DSL**, quando a DSL está contida na linguagem hospedeira.
  - ▶ Exemplo: SQL.

▶ 79

79

## Nível Externo (3/4)

### ▶ Conceitos:

- ▶ Vistas
  - ▶ É uma definição dos dados a que cada utilizador quer ou pode ter acesso
    - Normalmente só pode ver parte dos dados de uma base de dados
    - Uma das formas poderosas de implementar a segurança numa base de dados
- ▶ Esquema
  - ▶ São as instruções que definem uma vista (p. e. declarar que o utilizador X, só pode ver o número e o nome mas não, pode ver o salário em tabelas da base de dados)

▶ 80

80

## Nível Externo (4/4)

- ▶ Execução de comandos SQL:
  - ▶ DBMS é visto como um “interpretador de comandos” para cada comando SQL
    - ▶ Analisa-o, verifica os esquemas estabelecidos para as vistas externas, ...
    - ▶ Executa as operações necessárias à satisfação do comando de acordo com as condições definidas.
  - ▶ Os comandos são normalmente compilados antes de serem efectuados, passando por muitas fases de otimização (uso de álgebra relacional, cálculo relacional, ....) antes de serem processados.

▶ 81

81

## Nível Conceptual

- ▶ Faz a ligação entre o nível externo e o físico.
- ▶ Visão global da base de dados

▶ 82

82